

EasyGIS Remote Sensing Software

Plataforma de Sensoriamento Remoto para
Agricultura de Precisão

Trabalho Prático 1: Concepção, Requisitos e Arquitetura
Disciplina: Estágio Supervisionado – Fábrica de Software

Luis Gustavo Olimpio Lauan Matheus João Leonardi

João Vinícius Castello Vinícius Henrique Lucas Benevinuto

José Melquíades Sammuel Moura Saraiva

05 de março de 2026

Sumário

1	Introdução	5
2	Metodologia	5
2.1	Framework Scrum	5
2.2	Papéis Scrum	6
2.3	Artefatos Scrum	6
2.4	Cerimônias Scrum	7
2.5	Estimativa e Velocidade	7
3	Visão do Produto	8
3.1	Contexto e Problema	8
3.2	Solução Proposta	8
3.3	Usuários-Alvo	9
3.4	Diferenciais Competitivos	9
4	Backlog do Produto	10
4.1	Épico 1: Gestão de Talhões Agrícolas	10
4.2	Épico 2: Processamento de Índices Espectrais	10
4.3	Épico 3: Visualização e Interação com Mapa	10
4.4	Épico 4: Laboratório de Experimentos	11
4.5	Épico 5: Classificação de Culturas	11
4.6	Épico 6: Gestão de Dados Sentinel-2	11
4.7	Priorização MoSCoW	11
5	Especificação de Requisitos de Software	11
5.1	Requisitos Funcionais	12
5.1.1	Gestão de Dados Geoespaciais (RF-01 a RF-04)	12
5.1.2	Processamento Espectral (RF-05 a RF-12)	12
5.1.3	Visualização e Interação (RF-13 a RF-15)	12
5.1.4	Experimentos e Classificação (RF-16 a RF-20)	12

5.2	Requisitos Não Funcionais	13
5.3	Matriz de Rastreabilidade	13
6	Arquitetura do Sistema	14
6.1	Visão Geral da Arquitetura	14
6.2	Diagrama de Componentes	14
6.3	Diagrama de Implantação	15
6.4	Diagrama de Sequência: Cálculo de Índice NDVI	15
6.5	Diagrama de Sequência: Classificação de Culturas	16
6.6	Decisões Arquiteturais	16
6.7	Stack Tecnológica	17
7	Protótipo de Interface	17
7.1	Dashboard Principal	18
7.2	Página de Classificação	18
7.3	Componentes Reutilizáveis	18
7.4	Identidade Visual	19
8	Plano de Projeto	19
8.1	Equipe e Papéis Scrum	19
8.2	Fase 1: Concepção e Design (TP1, 08/02 a 05/03)	20
8.3	Fase 2: Implementação e Entrega do MVP (TP2, 06/03 a 11/05)	20
8.3.1	Sprint 1: Fundação (06/03 a 19/03)	21
8.3.2	Sprint 2: Funcionalidades Core (20/03 a 02/04)	21
8.3.3	Sprint 3: Funcionalidades Avançadas (03/04 a 16/04)	21
8.3.4	Sprint 4: Experimentos e Classificação (17/04 a 30/04)	22
8.3.5	Sprint 5: Testes, Refinamento e Entrega (01/05 a 11/05)	22
8.4	Marcos do Projeto	22
8.5	Gestão de Riscos	22
8.6	Definição de Pronto	23
8.7	Definição de Preparado	23

8.8	Gestão de Mudanças	23
8.9	Monitoramento e Comunicação	24
9	Itens Complementares para o TP2	24
10	Considerações Finais	24
	Referências	26

1 Introdução

Este documento consolida os artefatos de concepção, requisitos e arquitetura do **EasyGIS Remote Sensing Software**, plataforma web de sensoriamento remoto voltada para agricultura de precisão. O trabalho corresponde ao **Trabalho Prático 1** da disciplina de Estágio Supervisionado (Fábrica de Software) do ICEV, elaborado entre **8 de fevereiro** e **5 de março de 2026**, seguindo a metodologia Scrum detalhada na Seção 2, sob orientação do professor, que exerce o papel de Product Owner.

A disciplina organiza-se em duas entregas. O TP1, objeto deste documento, abrangeu a fase de concepção: visão do produto, backlog organizado em épicos e histórias de usuário, especificação formal de requisitos funcionais e não funcionais, arquitetura projetada com diagramas UML, protótipos de interface e plano de projeto com cronograma, atribuições e gestão de riscos. O **Trabalho Prático 2**, com entrega prevista para **11 de maio de 2026**, compreenderá o desenvolvimento, os testes e a entrega do MVP funcional, conforme o plano de implementação descrito na Seção 8.

O software processará imagens multiespectrais do satélite Sentinel-2 da Agência Espacial Europeia (ESA) sobre talhões agrícolas delimitados por arquivos KML, produzindo índices de vegetação, experimentos de processamento de imagem e classificações automatizadas de culturas. Os resultados serão apresentados em interface web interativa com mapa bidimensional e visualização tridimensional.

2 Metodologia

2.1 Framework Scrum

O desenvolvimento do EasyGIS Remote Sensing Software adota o framework Scrum conforme descrito no *Scrum Guide* (Schwaber; Sutherland, 2020), com adaptações ao contexto acadêmico da disciplina de Estágio Supervisionado. O Scrum fundamenta-se em três pilares: transparência, inspeção e adaptação. A transparência é assegurada pelo backlog do produto visível a todos os membros e pelo acompanhamento contínuo do progresso em cada sprint. A inspeção ocorre nas cerimônias de revisão e retrospectiva, nas quais o time avalia o incremento produzido e o próprio processo de trabalho. A adaptação manifesta-se no refinamento contínuo do backlog e na reordenação de prioridades pelo Product Owner a cada ciclo.

A duração dos sprints varia conforme a fase do projeto. Na Fase 1 (TP1), foram adotados sprints de uma semana, totalizando quatro ciclos entre 8 de fevereiro e 5 de março de 2026. Essa duração reduzida justifica-se pelo cronograma compacto da fase

de concepção e pela necessidade de ciclos de feedback mais frequentes com o Product Owner. Na Fase 2 (TP2), os sprints terão duração de duas semanas, mais próxima do padrão recomendado pelo Scrum Guide, em razão do maior volume de trabalho técnico de implementação.

2.2 Papéis Scrum

Três papéis compõem a estrutura organizacional do projeto:

- **Product Owner (PO):** exercido pelo professor da disciplina, é responsável por maximizar o valor do produto, manter e priorizar o backlog, definir critérios de aceitação das histórias de usuário e validar os incrementos entregues ao final de cada sprint. O PO participa das cerimônias de Sprint Planning e Sprint Review, garantindo alinhamento entre as entregas e os objetivos pedagógicos da disciplina.
- **Scrum Master (SM):** exercido por Luis Gustavo Olimpio, que acumula a função de Tech Lead. O Scrum Master facilita as cerimônias, remove impedimentos que bloqueiam o progresso do time, protege a equipe de interferências externas e promove a adoção das práticas ágeis. No contexto do projeto, o SM também coordena as decisões arquiteturais e a integração entre os módulos de frontend e backend.
- **Development Team:** composto pelos sete demais integrantes (Lauan Matheus, João Leonardi, João Vinícius Castello, Vinícius Henrique, Lucas Benevenuto, José Melquíades e Sammuel Moura Saraiva), é auto-organizado e multifuncional. Cada membro possui competências específicas (frontend, backend, geoespacial, QA), porém todos contribuem para o objetivo da sprint. A auto-organização significa que o time decide internamente como transformar itens do backlog em incrementos funcionais, sem imposição externa de tarefas individuais.

2.3 Artefatos Scrum

O processo de desenvolvimento produz e mantém três artefatos principais:

- **Product Backlog:** lista ordenada de todas as funcionalidades, melhorias e correções desejadas para o produto. Contém 35 histórias de usuário organizadas em seis épicos, priorizadas pelo método MoSCoW e refinadas continuamente pelo PO em colaboração com o Development Team. Cada item possui descrição no formato canônico ágil, prioridade (alta, média, baixa) e critérios de aceitação.
- **Sprint Backlog:** subconjunto do Product Backlog selecionado para cada sprint durante a cerimônia de Sprint Planning. Inclui as histórias de usuário comprometidas

e o plano de trabalho para alcançá-las. O Sprint Backlog pertence exclusivamente ao Development Team e pode ser ajustado durante a sprint, desde que o objetivo da sprint seja preservado.

- **Incremento:** resultado tangível e potencialmente entregável ao final de cada sprint. Cada incremento deve atender à Definição de Pronto (Definition of Done) estabelecida pela equipe, acumulando-se aos incrementos anteriores para formar o produto em construção.

2.4 Cerimônias Scrum

Quatro cerimônias estruturam o ritmo de trabalho em cada sprint:

- **Sprint Planning:** realizada no primeiro dia de cada sprint, com duração de até 1 hora. O PO apresenta os itens de maior prioridade do backlog, o Development Team estima o esforço necessário e seleciona os itens que compõem o Sprint Backlog. O resultado é o Sprint Goal, objetivo claro e conciso que orienta o trabalho da sprint.
- **Daily Scrum:** reunião diária de no máximo 15 minutos, na qual cada membro responde a três perguntas: o que fez desde a última reunião, o que pretende fazer até a próxima e quais impedimentos enfrenta. No contexto do projeto, a Daily Scrum ocorre de forma assíncrona via canal de comunicação da equipe nos dias sem encontro presencial.
- **Sprint Review:** realizada no último dia da sprint, com duração de até 30 minutos. O Development Team demonstra o incremento funcional ao PO, que avalia se os critérios de aceitação foram atendidos e fornece feedback. Itens não concluídos retornam ao Product Backlog para repriorização.
- **Sprint Retrospective:** realizada após a Sprint Review, com duração de até 20 minutos. A equipe discute o que funcionou bem, o que pode ser melhorado e quais ações concretas serão adotadas na próxima sprint. O foco é a melhoria contínua do processo, não do produto.

2.5 Estimativa e Velocidade

A estimativa de esforço das histórias de usuário utiliza a técnica de *story points* com a sequência de Fibonacci (1, 2, 3, 5, 8, 13). Histórias com estimativa superior a 8 pontos são decompostas em itens menores para reduzir incerteza e facilitar o acompanhamento. A velocidade do time, definida como a soma dos story points entregues por sprint, será mensurada a partir da primeira sprint de implementação e utilizada para calibrar o planejamento das sprints seguintes.

O backlog foi priorizado pelo método **MoSCoW**, que classifica cada item em quatro categorias: *Must Have* (essencial para o MVP), *Should Have* (agrega valor significativo), *Could Have* (desejável, mas dispensável) e *Won't Have* (fora do escopo da versão atual). Essa classificação, combinada com a estimativa em story points, permite ao PO tomar decisões informadas sobre o que incluir em cada sprint.

3 Visão do Produto

3.1 Contexto e Problema

O Brasil figura entre os maiores produtores agrícolas do mundo, com aproximadamente 66 milhões de hectares de área plantada. A agricultura de precisão, que emprega tecnologias para gerenciar a variabilidade espacial e temporal das lavouras, tem apresentado crescimento significativo no país, porém ainda enfrenta barreiras substanciais de adoção. Entre os principais obstáculos estão o elevado custo de ferramentas GIS comerciais como ArcGIS e ENVI, cujo licenciamento pode ultrapassar milhares de reais anuais; a complexidade técnica de softwares profissionais de sensoriamento remoto, que exigem treinamento especializado; e a inexistência de soluções integradas que reúnam processamento de imagem, análise espectral e visualização em plataforma única e acessível.

O programa Copernicus da ESA, por sua vez, disponibiliza gratuitamente imagens Sentinel-2 com resolução espacial de 10 metros e tempo de revisita de 5 dias, o que configura oportunidade singular para democratizar o acesso à agricultura de precisão. A utilização direta dessas imagens, no entanto, demanda conhecimentos especializados em geoprocessamento, manipulação de arquivos JP2 e interpretação de bandas espectrais, habilidades ausentes na formação da maioria dos produtores rurais e técnicos agrícolas.

O problema central reside em monitorar a saúde de culturas agrícolas de modo rápido e acessível, sem depender de vistorias presenciais dispendiosas ou de softwares proprietários de alto custo. As consequências desse cenário são decisões tardias ou desinformadas sobre irrigação, aplicação de insumos e manejo de pragas, com impacto direto na produtividade e nos custos operacionais.

3.2 Solução Proposta

O EasyGIS Remote Sensing Software foi concebido como plataforma web gratuita e de código aberto que permitirá a usuários sem conhecimento avançado em geoprocessamento carregar limites de talhões agrícolas, processar imagens multiespectrais de satélite e gerar mapas de índices vegetativos acompanhados de estatísticas descritivas, tudo por meio de

interface intuitiva acessível via navegador.

A solução será composta por dois módulos. O **backend**, a ser desenvolvido em Python com FastAPI, concentrará o processamento geoespacial: leitura de bandas espectrais JP2 via rasterio, recorte de imagens ao polígono do talhão mediante reprojeção de coordenadas com pyproj e mascaramento espacial, cálculo de índices de vegetação (NDVI, EVI, SAVI) com NumPy, execução de experimentos de visão computacional com SciPy e OpenCV, e classificação de culturas com scikit-learn. O **frontend**, projetado com Next.js 16, React 19 e TypeScript, proverá interface com mapa interativo Leaflet sobre imagens de satélite, painel de controle para seleção de análises, visualização tridimensional via Three.js e página dedicada à classificação de culturas.

3.3 Usuários-Alvo

Quatro perfis de usuário orientam as decisões de projeto. O **produtor rural**, proprietário ou gestor de propriedade agrícola, necessita monitorar a saúde das lavouras, identificar áreas de estresse hídrico ou nutricional e otimizar a aplicação de insumos. O **agrônomo**, profissional técnico responsável pelo manejo, busca gerar mapas de variabilidade para prescrição de insumos e analisar o vigor vegetativo ao longo do ciclo produtivo. O **pesquisador acadêmico**, inserido no campo do sensoriamento remoto ou da agronomia, deseja experimentar algoritmos de processamento de imagem sobre dados reais e comparar índices espectrais. O **estudante** de agronomia, engenharia agrícola ou ciência da computação utiliza a plataforma para aprender conceitos de sensoriamento remoto de forma prática e visual.

3.4 Diferenciais Competitivos

Seis características distinguem a plataforma das soluções existentes no mercado: (i) é gratuita e de código aberto, sem custos de licenciamento; (ii) oferece interface web intuitiva que dispensa instalação de software GIS desktop; (iii) inclui visualização 3D interativa como diferencial pedagógico para compreensão da variabilidade espacial; (iv) disponibiliza laboratório de experimentos com 13 algoritmos de visão computacional aplicáveis a dados reais de satélite; (v) integra classificação de culturas, da ingestão do dado à área classificada em hectares, em plataforma única; e (vi) tem foco no contexto agropecuário brasileiro, com tipos de cultura (soja, milho, café, cana-de-açúcar) e geolocalização padrão no estado do Piauí.

4 Backlog do Produto

O backlog do produto está organizado em seis épicos que agrupam as funcionalidades por domínio de negócio. Cada épico contém histórias de usuário no formato canônico ágil (*como* [persona], *quero* [ação], *para que* [benefício]), classificadas por prioridade (alta, média ou baixa). Ao todo, foram identificadas 35 histórias de usuário, priorizadas pelo Product Owner para implementação ao longo das sprints planejadas.

4.1 Épico 1: Gestão de Talhões Agrícolas

O primeiro épico abrange a definição e o gerenciamento das áreas agrícolas objeto de análise. A funcionalidade de maior prioridade é a importação de limites de talhões a partir de arquivos KML exportados do Google Earth (US-01), seguida pela visualização dos polígonos sobre imagem de satélite no mapa interativo (US-02). O sistema deverá permitir o desenho de novos polígonos diretamente no mapa (US-03), a exibição da área em hectares de cada talhão (US-04), a persistência dos talhões desenhados (US-05) e a busca de localizações por endereço (US-06).

4.2 Épico 2: Processamento de Índices Espectrais

O núcleo funcional do sistema concentra-se neste épico, que compreende o cálculo e a visualização de índices de vegetação. As histórias de maior prioridade são o cálculo do NDVI para avaliação de saúde vegetal (US-07), do EVI para análise com correção atmosférica (US-08) e a geração de estatísticas descritivas por índice (US-11). Contemplam-se ainda a visualização de composições em cor verdadeira e falsa-cor (US-09), a exibição individual das 13 bandas espectrais do Sentinel-2 (US-10), a geração de histogramas de distribuição (US-12), a aplicação de suavização gaussiana (US-13) e a integração dos índices NDWI e SAVI ao endpoint principal (US-14).

4.3 Épico 3: Visualização e Interação com Mapa

A experiência de exploração visual dos dados constitui o foco deste épico. A sobreposição do resultado espectral ao mapa de satélite (US-16) e a legenda com escala de cores e guia de interpretação (US-20) são os itens prioritários. A alternância entre camadas de satélite e de ruas (US-17), a exibição do nível de estresse ao clicar no mapa (US-18) e a visualização 3D interativa do terreno espectral (US-19) complementam o conjunto.

4.4 Épico 4: Laboratório de Experimentos

Voltado a pesquisadores e estudantes, este épico prevê 13 algoritmos de processamento de imagem em cinco categorias: filtros de suavização (US-21), detectores de borda (US-22), operações morfológicas (US-23), segmentação por limiar (US-24) e ajuste interativo de parâmetros via sliders (US-25). A exportação de resultados para relatórios (US-26) complementa o conjunto.

4.5 Épico 5: Classificação de Culturas

A classificação automatizada das áreas agrícolas é o foco deste épico. A separação por limiar de NDVI em zonas de vigor (US-27), a quantificação de área por classe em hectares (US-29) e a exibição do resultado como overlay colorido no mapa (US-32) configuram os itens de alta prioridade. A classificação não supervisionada por K-Means (US-28), a seleção de índices como features (US-30) e a classificação supervisionada com Random Forest (US-31) completam as funcionalidades previstas.

4.6 Épico 6: Gestão de Dados Sentinel-2

O sexto épico contempla a listagem de produtos disponíveis (US-33), a verificação de saúde do sistema (US-34) e o download automático de imagens Sentinel-2 (US-35), este último classificado como *won't have* para a primeira versão.

4.7 Priorização MoSCoW

A priorização seguiu o método MoSCoW. Os itens **Must Have** (10 histórias) correspondem ao MVP funcional: importação de KML, mapa, NDVI, EVI, estatísticas, overlay, legenda, classificação por limiar e exibição de resultados. Os itens **Should Have** (17 histórias) agregam valor significativo: composições RGB, bandas individuais, visualização 3D, experimentos e K-Means. Os itens **Could Have** (7 histórias) representam melhorias desejáveis, enquanto o download automático foi o único **Won't Have**.

5 Especificação de Requisitos de Software

A especificação de requisitos segue a norma IEEE 830-1998, adaptada ao contexto do projeto. Os requisitos estão categorizados em funcionais (RF) e não funcionais (RNF), com rastreabilidade direta às histórias de usuário do backlog.

5.1 Requisitos Funcionais

Foram especificados 20 requisitos funcionais, organizados em quatro grupos temáticos. Todos descrevem comportamentos que o sistema deverá apresentar quando implementado.

5.1.1 Gestão de Dados Geoespaciais (RF-01 a RF-04)

O sistema deverá ler arquivos `.kml` do diretório de dados, extraindo polígonos de talhões com coordenadas geográficas, bounding box, área aproximada em hectares e centroide (RF-01). Os polígonos serão renderizados sobre o mapa Leaflet com diferenciação visual: azul para o talhão selecionado, verde para os demais (RF-02). Ferramentas de desenho permitirão criar novos talhões diretamente no mapa, adicionados à lista com ID gerado automaticamente (RF-03). O backend listará todos os produtos Sentinel-2 disponíveis no diretório de imagens, identificados pelo padrão `S2*_MSIL2A_*.SAFE` (RF-04).

5.1.2 Processamento Espectral (RF-05 a RF-12)

O cálculo do NDVI (RF-05) utilizará as bandas B08 (NIR, 10m) e B04 (Red, 10m) segundo a fórmula $(NIR - Red)/(NIR + Red)$, com resultado limitado ao intervalo $[-1, 1]$. O EVI (RF-06) incorporará a banda B02 (Blue) e aplicará a expressão $2,5 \times (NIR - Red)/(NIR + 6 \times Red - 7,5 \times Blue + 1)$. Composições em cor verdadeira (RF-07) e falsa-cor (RF-08) serão geradas combinando bandas em 10m de resolução. A visualização individual de cada uma das 13 bandas do Sentinel-2 em resoluções nativas, a saber 10m (B02, B03, B04, B08), 20m (B05 a B07, B8A, B11, B12) e 60m (B01, B09, B10), é contemplada pelo RF-09. Para cada cálculo, o sistema retornará estatísticas descritivas (RF-10) e histograma de 50 bins (RF-11). A suavização gaussiana com $\sigma = 1,5$ será aplicada opcionalmente (RF-12).

5.1.3 Visualização e Interação (RF-13 a RF-15)

Os resultados serão exibidos como overlay de imagem sobre o bounding box do talhão no mapa (RF-13). Para o NDVI, popups interativos classificarão o estresse em quatro níveis: severo ($< 0,2$), moderado ($0,2-0,4$), leve ($0,4-0,6$) e saudável ($\geq 0,6$) (RF-14). A visualização 3D renderizará os valores de pixel como elevação em um terreno tridimensional interativo com rotação, zoom e deslocamento (RF-15).

5.1.4 Experimentos e Classificação (RF-16 a RF-20)

O laboratório de experimentos (RF-16) disponibilizará 13 algoritmos sobre a banda NIR, organizados em cinco categorias: filtros (gaussiano, mediana, bilateral), detecção de bor-

das (Sobel, Canny, laplaciano), equalização de histograma, operações morfológicas (erosão, dilatação, abertura, fechamento) e segmentação por limiar (binário, Otsu, adaptativo). A classificação por limiar de NDVI (RF-17) separará os pixels em três zonas de vigor com cálculo de área em hectares. A classificação não supervisionada (RF-18) agrupará pixels em 2 a 6 classes via K-Means. A legenda de classificação (RF-19) exibirá nome, cor, porcentagem e área de cada classe. O endpoint de verificação de saúde (RF-20) retornará o status do backend e a disponibilidade do diretório de produtos.

5.2 Requisitos Não Funcionais

Oito requisitos não funcionais foram definidos para orientar a qualidade do sistema:

- **Desempenho (RNF-01):** O cálculo de índice espectral para um talhão típico deve completar em até 30 segundos. A renderização do mapa 2D deve manter taxa acima de 30 FPS.
- **Usabilidade (RNF-02):** A interface deve ser utilizável sem treinamento prévio por um agrônomo com conhecimento básico de informática. Parâmetros de experimentos são ajustáveis via sliders com valores padrão razoáveis.
- **Compatibilidade (RNF-03):** Navegadores Chrome 90+, Firefox 80+, Edge 90+ e Safari 15+. Backend compatível com Linux, macOS e Windows (via WSL). Python 3.12+ e Node.js 18+.
- **Confiabilidade (RNF-04):** Mensagens de erro claras (HTTP 400/500), feedback visual durante processamento e tratamento gracioso de KML malformados.
- **Manutenibilidade (RNF-05):** Código organizado em módulos (API, processador, componentes, utilitários), com tipagem estática (TypeScript, Pydantic) e separação de responsabilidades.
- **Portabilidade (RNF-06):** Sem dependência de banco de dados. Gerenciamento de dependências via `uv` (Python) e `npm` (Node.js).
- **Segurança (RNF-07):** CORS restrito a `localhost:3000`. Sem autenticação (sistema local single-user).
- **Escalabilidade (RNF-08):** Projetado para uso single-user. Processamento limitado pela RAM disponível.

5.3 Matriz de Rastreabilidade

A Tabela 1 apresenta a correspondência entre requisitos funcionais e histórias de usuário, assegurando cobertura completa e bidirecional.

Tabela 1: Rastreabilidade entre requisitos funcionais e histórias de usuário.

Requisito	Histórias	Domínio
RF-01, RF-02, RF-03	US-01, US-02, US-03, US-04	Gestão de Talhões
RF-04	US-33	Dados Sentinel-2
RF-05, RF-06	US-07, US-08	Índices Espectrais
RF-07, RF-08, RF-09	US-09, US-10	Composições e Bandas
RF-10, RF-11, RF-12	US-11, US-12, US-13	Estatísticas
RF-13, RF-14, RF-15	US-16, US-18, US-19	Visualização
RF-16	US-21 a US-25	Experimentos
RF-17, RF-18, RF-19	US-27 a US-30, US-32	Classificação
RF-20	US-34	Saúde do Sistema

6 Arquitetura do Sistema

A arquitetura projetada para o EasyGIS Remote Sensing Software segue o padrão cliente-servidor com dois processos independentes comunicando-se via API REST. Os diagramas UML a seguir descrevem a estrutura, o comportamento e a implantação projetados para o sistema.

6.1 Visão Geral da Arquitetura

O sistema operará como dois processos na máquina local do usuário. O frontend, servidor de desenvolvimento Next.js na porta 3000, renderizará a interface no navegador e realizará chamadas HTTP ao backend. O backend, servidor Uvicorn/FastAPI na porta 8000, processará as requisições geoespaciais e retornará resultados em JSON com imagens codificadas em base64. Ambos os processos acessarão o sistema de arquivos local: o frontend lerá arquivos KML de talhões, ao passo que o backend lerá imagens Sentinel-2 em formato JP2.

A separação em dois processos é motivada por duas razões: (i) impedir que o processamento pesado de imagens em Python bloqueie a interface do usuário; e (ii) viabilizar o desenvolvimento paralelo pelas equipes de frontend e backend.

6.2 Diagrama de Componentes

A Figura 1 apresenta o diagrama de componentes do sistema, mostrando os módulos principais e suas dependências.

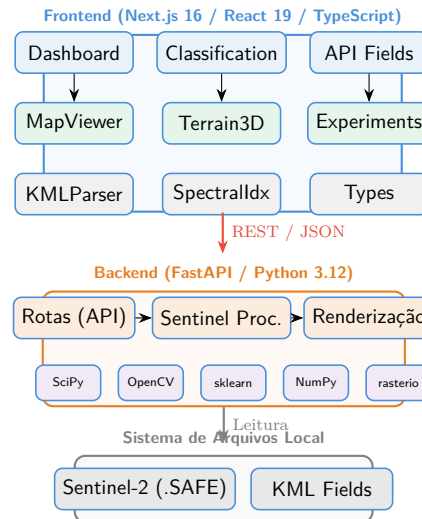


Figura 1: Diagrama de componentes do EasyGIS Remote Sensing Software.

6.3 Diagrama de Implantação

A Figura 2 ilustra a topologia de implantação do sistema. Todos os nós executam na mesma máquina local, com três serviços externos acessados exclusivamente para leitura de tiles de mapa.

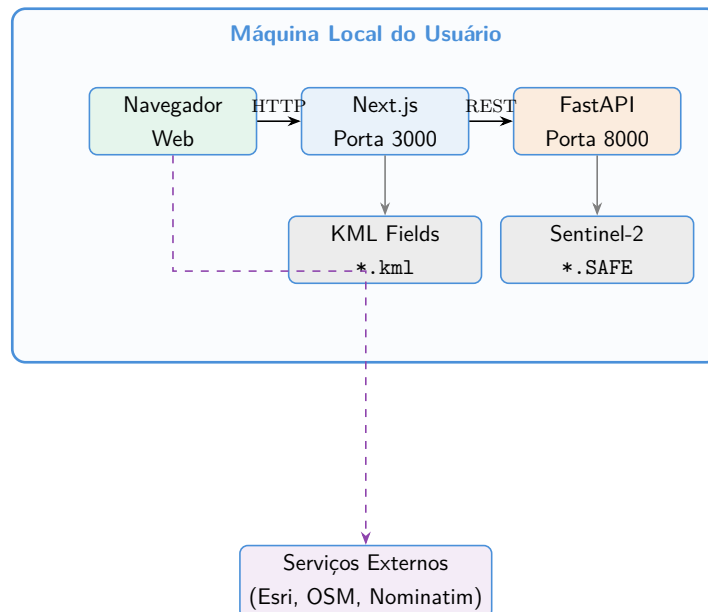


Figura 2: Diagrama de implantação do sistema.

6.4 Diagrama de Sequência: Cálculo de Índice NDVI

A Figura 3 detalha o fluxo de interação para o cálculo do NDVI, desde a seleção do talhão pelo usuário até a exibição do resultado no mapa.

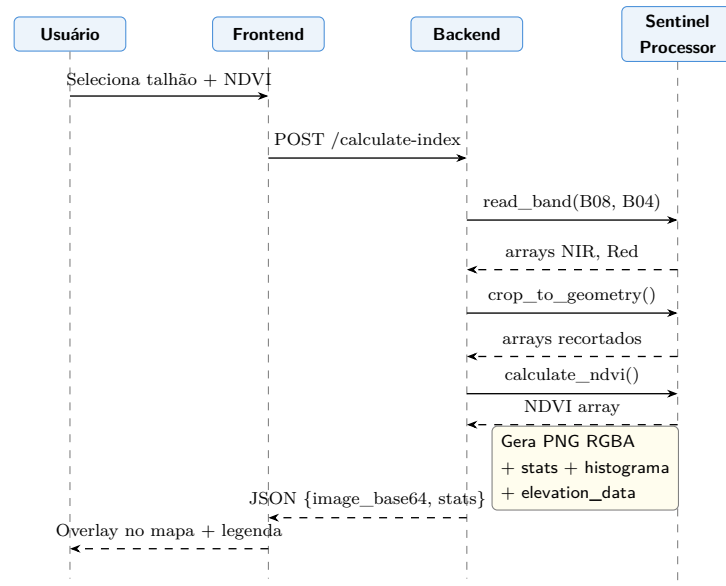


Figura 3: Diagrama de sequência para o cálculo de NDVI.

6.5 Diagrama de Sequência: Classificação de Culturas

A Figura 4 apresenta o fluxo de classificação não supervisionada, em que os índices selecionados são calculados, empilhados como features e submetidos ao algoritmo K-Means.

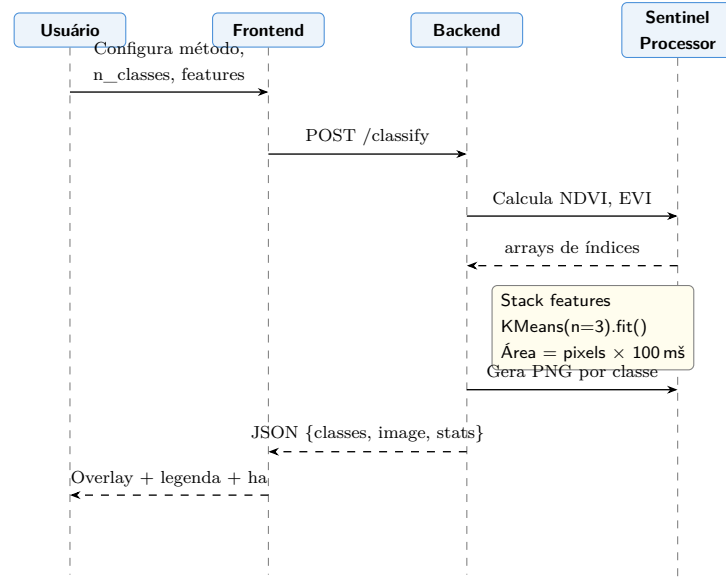


Figura 4: Diagrama de sequência para classificação de culturas.

6.6 Decisões Arquiteturais

Cinco decisões arquiteturais fundamentais orientam o projeto:

1. **Arquitetura em dois processos:** a separação entre frontend e backend viabilizará

o desenvolvimento paralelo e impedirá que o processamento pesado de imagens bloqueie a interface.

2. **Transferência de imagens via base64:** codificar imagens PNG como strings base64 dentro do payload JSON simplificará a API ao custo de cerca de 33% de overhead no tamanho da transferência, eliminando a necessidade de servir arquivos estáticos.
3. **Processamento em memória sem persistência:** a ausência de banco de dados reduzirá a complexidade de infraestrutura para a primeira versão; cada análise será independente e *stateless*.
4. **Reprojeção dinâmica de coordenadas:** o frontend trabalhará em EPSG:4326 (WGS84), enquanto as imagens Sentinel-2 utilizam projeções UTM; a reprojeção automática via pyproj garantirá compatibilidade sem expor complexidade ao usuário.
5. **KML como formato de entrada:** amplamente suportado pelo Google Earth e QGIS, o formato facilita a interoperabilidade e a adoção por usuários sem ferramentas especializadas.

6.7 Stack Tecnológica

A Tabela 2 consolida as tecnologias selecionadas para o projeto.

Tabela 2: Stack tecnológica do EasyGIS Remote Sensing Software.

Camada	Tecnologia	Função
Frontend	Next.js 16 / React 19	Framework web com SSR e App Router
	TypeScript 5	Tipagem estática
	Tailwind CSS 4	Estilização utilitária
	Leaflet 1.9	Mapa interativo 2D
	Three.js 0.180	Renderização 3D (WebGL)
	shadcn/ui + Radix	Componentes de interface
Backend	Python 3.12	Linguagem principal
	FastAPI 0.115+	Framework REST
	rasterio 1.4+	Leitura de JP2 geoespaciais
	NumPy 2.0+	Processamento numérico
	SciPy / OpenCV	Filtros e visão computacional
	scikit-learn	K-Means clustering

7 Protótipo de Interface

A interface do sistema foi prototipada com duas telas principais, ambas seguindo o padrão de layout dividido entre painel de controle lateral e área de visualização. Esta seção

descreve a organização visual, os fluxos de interação e os componentes projetados.

7.1 Dashboard Principal

A tela principal ocupará toda a viewport do navegador e divide-se em três zonas: barra de ícones de navegação (48px de largura), painel lateral de controle (384px fixos) e área de mapa ou visualização 3D ocupando o espaço restante.

O painel lateral organizar-se-á em duas abas. A aba **Analytics** concentrará os controles de análise espectral: seleção de talhão via dropdown, abas de análise rápida (Zone/NDVI, Productivity/EVI, Nitrogen/SAVI), toggle de suavização, botão de execução, badges de estatísticas (média, mínimo, máximo), toggle de visualização 3D, legenda com gradiente de cores e guia de interpretação, e seletores para os 19 tipos de índice disponíveis (5 de vegetação, 2 composições RGB, 12 bandas individuais). A aba **Research** apresentará menu accordion com cinco categorias de experimentos totalizando 13 algoritmos, área de experimentos recentes e botões de ação rápida.

A área principal alternará entre o mapa Leaflet 2D, dotado de camadas de satélite Esri e ruas OpenStreetMap, ferramentas de desenho para polígonos e retângulos, overlay de resultados, popups de estresse NDVI e busca de endereço, e a visualização Three.js 3D, na qual os valores dos pixels serão renderizados como elevação em terreno texturizado com iluminação de três pontos e controles orbitais.

7.2 Página de Classificação

A segunda tela empregará layout 50/50. O painel esquerdo conterá controles de seleção de talhão, abas para método de classificação (supervisionada, não supervisionada, limiar), seletores condicionais de tipo de cultura ou número de classes, checkboxes de features (NDVI, EVI, SAVI) e botão de execução. Após a classificação, um painel de resultados exibirá área total, área classificada e legenda colorida com porcentagem e hectares por classe. O painel direito apresentará o mapa com overlay colorido de classificação sobre a imagem de satélite.

7.3 Componentes Reutilizáveis

O design system basear-se-á em componentes shadcn/ui (Button, Select, Tabs, Dialog, Slider, Switch, Label), complementados por cinco componentes customizados: **MapView** para o mapa Leaflet, **Terrain3DViewer** para visualização tridimensional, **ExperimentMenu** para o accordion de algoritmos, **ExperimentDialog** para configuração de parâmetros via sliders dinâmicos e **IndexLegend** para legenda com gradiente de cores e fórmula do índice.

7.4 Identidade Visual

A interface adotará tema escuro com fundo #09090b (zinc-950), cards em #18181b (zinc-900), cor primária verde #22c55e para ações e indicadores positivos, azul #3b82f6 para destaques de seleção, texto principal em #fafafa e texto secundário em #a1a1aa. O gradiente de NDVI seguirá uma escala de seis faixas, do marrom (solo exposto, valores negativos) ao verde escuro (vegetação densa, acima de 0,6), conforme representado na Tabela 3.

Tabela 3: Gradiente de cores para visualização do NDVI.

Faixa NDVI	Cor	Interpretação
−1,0 a −0,2	 Marrom	Água / solo exposto
−0,2 a 0,0	 Marrom claro	Solo
0,0 a 0,2	 Dourado	Vegetação escassa
0,2 a 0,4	 Amarelo-verde	Vegetação moderada
0,4 a 0,6	 Verde lima	Vegetação saudável
0,6 a 1,0	 Verde escuro	Vegetação densa

8 Plano de Projeto

O projeto abrange duas fases distintas, correspondentes aos dois trabalhos práticos da disciplina. A **Fase 1 (TP1)**, de 8 de fevereiro a 5 de março de 2026, compreendeu a concepção, o levantamento de requisitos, o design da arquitetura, a elaboração dos protótipos de interface e a redação da documentação. A **Fase 2 (TP2)**, de 6 de março a 11 de maio de 2026, abrangerá a implementação, os testes e a entrega do MVP funcional, organizada em cinco sprints de duas semanas. Cada sprint possui um *Sprint Goal* explícito, definido na cerimônia de Sprint Planning em acordo entre o Product Owner e o Development Team.

8.1 Equipe e Papéis Scrum

A equipe Scrum é composta por oito integrantes, distribuídos conforme competências complementares. O papel de **Scrum Master** é exercido por Luis Gustavo Olimpio, que também atua como Tech Lead. Os demais compõem o **Development Team**.

Tabela 4: Composição da equipe e responsabilidades.

Membro	Papel Scrum	Responsabilidades Técnicas
Luis Gustavo Olimpio	Scrum Master / Tech Lead	Arquitetura geral, SentinelProcessor, rasterio
Lauan Matheus	Dev Team / Frontend Lead	Next.js, Leaflet, componentes React
João Leonardi	Dev Team / Backend	Endpoints FastAPI, Pydantic, classificação
João Vinícius Castello	Dev Team / Frontend	shadcn/ui, ExperimentDialog, responsividade
Vinícius Henrique	Dev Team / Full Stack	Three.js, Terrain3DViewer, elevação
Lucas Benevenuto	Dev Team / Geoespacial	KMLParser, bandas espectrais, testes
José Melquíades	Dev Team / QA	Testes, documentação, dependências
Sammuel Moura Saraiva	Dev Team / Backend	Endpoints FastAPI, integração

8.2 Fase 1: Concepção e Design (TP1, 08/02 a 05/03)

A primeira fase, concluída com a entrega deste documento, concentrou-se na definição conceitual e arquitetural do produto. As atividades realizadas foram:

- **Semana 1 (08/02 a 14/02):** Levantamento do problema e pesquisa de mercado. Definição dos usuários-alvo, identificação dos diferenciais competitivos e redação do Documento de Visão do Produto. Primeira sessão de elicitação de requisitos com o Product Owner.
- **Semana 2 (15/02 a 21/02):** Construção do Product Backlog com 35 histórias de usuário organizadas em seis épicos. Aplicação da priorização MoSCoW. Especificação dos 20 requisitos funcionais e 8 não funcionais segundo a norma IEEE 830. Elaboração da matriz de rastreabilidade.
- **Semana 3 (22/02 a 28/02):** Projeto da arquitetura cliente-servidor (FastAPI + Next.js). Elaboração dos diagramas UML: componentes, implantação e sequência. Definição da stack tecnológica. Seleção das decisões arquiteturais fundamentais.
- **Semana 4 (01/03 a 05/03):** Prototipação das interfaces (dashboard principal e página de classificação). Definição da identidade visual e do design system. Elaboração do plano de projeto com cronograma, riscos e cerimônias Scrum. Consolidação de toda a documentação no documento unificado.

8.3 Fase 2: Implementação e Entrega do MVP (TP2, 06/03 a 11/05)

A segunda fase organizará a implementação do software em cinco sprints de duas semanas, seguindo o cronograma descrito a seguir. Os Sprint Goals refletem a priorização MoSCoW

do backlog: as primeiras sprints entregam os itens *Must Have*, as intermediárias abordam os *Should Have* e as finais incorporam *Could Have* conforme a velocidade do time permitir.

8.3.1 Sprint 1: Fundação (06/03 a 19/03)

Sprint Goal: *Estabelecer a infraestrutura de desenvolvimento e validar a leitura de dados geoespaciais de ponta a ponta.*

Serão configurados o repositório Git com a estrutura de diretórios, o projeto Next.js com TypeScript e Tailwind CSS, e o projeto FastAPI com Uvicorn. Em paralelo, a classe `SentinelProcessor` será desenvolvida com leitura de bandas JP2 e recorte por geometria, o `KMLParser` será criado para extração de polígonos dos arquivos KML, a comunicação CORS entre frontend e backend será configurada e os tipos TypeScript serão definidos em `types/index.ts`. Ao término da sprint, espera-se que o backend leia imagens Sentinel-2 e o frontend renderize o mapa base com os talhões importados.

8.3.2 Sprint 2: Funcionalidades Core (20/03 a 02/04)

Sprint Goal: *Entregar o fluxo completo de cálculo e visualização de índices espectrais com estatísticas.*

No backend, serão desenvolvidos o endpoint `POST /calculate-index` com suporte a NDVI e EVI, o mecanismo de recorte por polígono (`crop_to_geometry`) com reprojeção automática de coordenadas e a renderização de PNG com gradiente de cores. No frontend, o componente `MapView` será criado com integração Leaflet, overlay de resultados, painel lateral com aba Analytics, seleção de talhão e índice, e componente `IndexLegend`. O módulo de estatísticas descritivas e histograma completará a sprint.

8.3.3 Sprint 3: Funcionalidades Avançadas (03/04 a 16/04)

Sprint Goal: *Ampliar as capacidades de visualização (3D, bandas, composições) e interação (desenho, busca).*

O componente `Terrain3DViewer` será desenvolvido com Three.js, incluindo geometria de plano deformada por elevação, texturização com a imagem de índice, iluminação de três pontos e controles orbitais. Composições RGB e falsa-cor serão adicionadas, assim como a visualização das 13 bandas individuais do Sentinel-2 em resolução nativa. As ferramentas de desenho via leaflet-draw, a busca de endereço via leaflet-geosearch, o popup interativo de estresse NDVI e a suavização gaussiana completarão esta sprint.

8.3.4 Sprint 4: Experimentos e Classificação (17/04 a 30/04)

Sprint Goal: *Entregar o laboratório de experimentos e a classificação de culturas.*

Serão desenvolvidos o endpoint de experimentos com 13 algoritmos de processamento de imagem, os componentes `ExperimentMenu` e `ExperimentDialog`, a aba Research no painel lateral, a página de classificação de culturas, o endpoint de classificação com suporte a K-Means, limiar de NDVI e classificação supervisionada, além do cálculo de área por classe em hectares.

8.3.5 Sprint 5: Testes, Refinamento e Entrega (01/05 a 11/05)

Sprint Goal: *Corrigir bugs, executar testes de integração e preparar o MVP para entrega final.*

A sprint final concentrar-se-á na estabilização do sistema. Serão realizados testes de integração end-to-end com dados reais Sentinel-2, correção de bugs identificados nas sprints anteriores, declaração de todas as dependências no `pyproject.toml`, implementação de estados de carregamento e mensagens de erro amigáveis, e redação da documentação técnica complementar (guia de instalação e manual do usuário). A Sprint Review final constituirá a demonstração do MVP ao Product Owner.

8.4 Marcos do Projeto

Tabela 5: Marcos (*milestones*) do projeto.

Marco	Data	Critério	Status
M0: Concepção (TP1)	05/03	Documentação completa entregue	Concluído
M1: Infraestrutura	19/03	Backend lendo JP2, frontend com mapa	Planejado
M2: MVP Funcional	02/04	NDVI no mapa com estatísticas	Planejado
M3: Features Completas	16/04	3D, bandas, desenho, busca	Planejado
M4: Produto Completo	30/04	Experimentos + classificação	Planejado
M5: Entrega Final (TP2)	11/05	MVP testado, documentado, estável	Planejado

8.5 Gestão de Riscos

A Tabela 6 apresenta os riscos identificados durante o planejamento do projeto, classificados por probabilidade e impacto, com as respectivas estratégias de mitigação previstas.

Tabela 6: Registro de riscos do projeto.

Risco	Prob.	Impacto	Mitigação
Imagens Sentinel-2 grandes demais para RAM	Média	Alto	Limitar tamanho de talhão
Dependências Python não declaradas	Alta	Médio	Incluir no <code>pyproject.toml</code>
Latência de processamento para talhões grandes	Média	Médio	Feedback de progresso
Navegadores sem suporte a WebGL	Baixa	Baixo	Fallback para mapa 2D
KML malformados causam falha	Baixa	Médio	Validação no parser

8.6 Definição de Pronto

Uma tarefa será considerada concluída quando atender simultaneamente a seis critérios: código implementado e funcional; aderência aos padrões do projeto (TypeScript com tipagem, Pydantic para validação); ausência de erros de console no frontend e de exceções não tratadas no backend; teste manual com dados reais (imagem Sentinel-2 e arquivo KML); revisão por ao menos um outro membro da equipe; e código versionado no repositório Git.

8.7 Definição de Preparado

Complementarmente, a *Definition of Ready* estabelece os critérios que uma história de usuário deve satisfazer antes de ser incluída em um Sprint Backlog: descrição no formato canônico ágil (*como/quero/para que*); critérios de aceitação claros e verificáveis; estimativa em story points atribuída pelo Development Team; dependências técnicas identificadas e resolvidas ou explicitamente aceitas; e aprovação do Product Owner quanto à prioridade.

8.8 Gestão de Mudanças

Alterações no escopo do projeto são gerenciadas por meio do refinamento contínuo do Product Backlog. Novas histórias de usuário, alterações de prioridade ou remoção de itens são propostas pelo PO ou pelo Development Team e discutidas nas sessões de *Backlog Refinement*, realizadas na metade de cada sprint. Mudanças no Sprint Backlog durante uma sprint em andamento são evitadas para preservar a estabilidade do Sprint Goal; caso um impedimento técnico torne um item inviável, o Scrum Master facilita a renegociação de escopo com o PO sem comprometer o objetivo da sprint.

O controle de versão via Git constitui o mecanismo primário de rastreabilidade de mudanças no código. Cada funcionalidade será desenvolvida em branch dedicada, submetida a revisão por pares (*code review*) e integrada à branch principal mediante *pull request*. Essa prática assegura que cada alteração seja auditável e reversível.

8.9 Monitoramento e Comunicação

O progresso de cada sprint será acompanhado por meio de três mecanismos. O *Sprint Burndown Chart* registrará a quantidade de story points restantes ao longo dos dias da sprint, permitindo identificar desvios em relação ao ritmo planejado. O *Kanban Board* (via GitHub Projects) organizará as tarefas em colunas *To Do*, *In Progress*, *In Review* e *Done*, oferecendo visibilidade instantânea do estado de cada item. As *Daily Scrums*, presenciais ou assíncronas, garantirão alinhamento diário e identificação precoce de impedimentos.

Ao final de cada sprint, a Sprint Review produzirá um registro do incremento demonstrado e do feedback do PO, enquanto a Sprint Retrospective gerará uma lista de ações de melhoria para o ciclo seguinte. Esses registros compõem o histórico processual do projeto e subsidiam a avaliação contínua da maturidade ágil da equipe.

9 Itens Complementares para o TP2

Além das 35 histórias de usuário do backlog principal, a equipe identificou eixos complementares que poderão ser incorporados ao escopo do TP2 conforme a velocidade do time e a priorização do Product Owner.

A **persistência de dados** prevê a adoção de banco de dados SQLite ou PostgreSQL para salvar talhões desenhados, resultados de análises e histórico por talhão. A **classificação supervisionada real** contempla o treinamento de modelo Random Forest sobre dados rotulados e a construção de pipeline de treinamento e inferência. A **qualidade de código** inclui testes unitários (pytest para backend, Jest para frontend), configuração de CI/CD com GitHub Actions e formatação automática (ruff para Python, ESLint para TypeScript). As **melhorias de experiência do usuário** abrangem estados de carregamento durante processamento e mensagens de erro amigáveis. Esses itens serão avaliados nas sessões de Backlog Refinement ao longo das sprints de implementação.

10 Considerações Finais

O Trabalho Prático 1 cumpriu integralmente o escopo da fase de concepção do EasyGIS Remote Sensing Software. Ao longo de quatro semanas, a equipe produziu os seis artefatos que compõem este documento: a visão do produto com análise de mercado e usuários-alvo; o backlog com 35 histórias de usuário organizadas em seis épicos e priorizadas pelo método MoSCoW; a especificação de 20 requisitos funcionais e 8 não funcionais com matriz de rastreabilidade; a arquitetura cliente-servidor projetada com diagramas UML de componentes, implantação e sequência; os protótipos de interface com identidade visual

e design system; e o plano de projeto com cronograma, cerimônias Scrum, gestão de riscos e definições de pronto e preparado.

A arquitetura em dois processos, FastAPI para processamento geoespacial e Next.js para interface interativa, foi dimensionada para viabilizar o desenvolvimento paralelo e a separação de responsabilidades. A integração planejada de rasterio, Leaflet e Three.js em plataforma única constituirá o principal diferencial técnico do sistema. A priorização MoSCoW assegura que os itens *Must Have* (importação de KML, NDVI, EVI, overlay, estatísticas, classificação por limiar) serão priorizados nas duas primeiras sprints de implementação do TP2.

Com a fase de concepção concluída, a equipe iniciará a implementação em 6 de março de 2026, seguindo o cronograma de cinco sprints que culmina na entrega do MVP funcional em 11 de maio de 2026, sob supervisão do Product Owner e coordenação do Scrum Master.

Referências

- [1] ESA. *Sentinel-2 Mission Guide*. Disponível em: <https://sentinel.esa.int/web/sentinel/missions/sentinel-2>. Acesso em: 5 mar. 2026.
- [2] OGC. *KML Standard*. Disponível em: <https://www.ogc.org/standard/kml/>. Acesso em: 5 mar. 2026.
- [3] Rasterio Contributors. *Rasterio Documentation*. Disponível em: <https://rasterio.readthedocs.io/>. Acesso em: 5 mar. 2026.
- [4] Ramírez, S. *FastAPI Documentation*. Disponível em: <https://fastapi.tiangolo.com/>. Acesso em: 5 mar. 2026.
- [5] Rouse, J. W. et al. Monitoring vegetation systems in the Great Plains with ERTS. *Third Earth Resources Technology Satellite-1 Symposium*, v. 1, p. 309–317, 1974.
- [6] Huete, A. R. et al. Overview of the radiometric and biophysical performance of the MODIS vegetation indices. *Remote Sensing of Environment*, v. 83, n. 1–2, p. 195–213, 2002.
- [7] Huete, A. R. A soil-adjusted vegetation index (SAVI). *Remote Sensing of Environment*, v. 25, n. 3, p. 295–309, 1988.
- [8] Pedregosa, F. et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, v. 12, p. 2825–2830, 2011.
- [9] Schwaber, K.; Sutherland, J. *The Scrum Guide*. 2020. Disponível em: <https://scrumguides.org/>. Acesso em: 5 mar. 2026.
- [10] Cohn, M. *Agile Estimating and Planning*. Upper Saddle River: Prentice Hall, 2005.